

SETUP INSTRUCTIONS:

BACKEND:

Step 1: Verify AWS CLI + credentials

Open terminal and run: `aws sts get-caller-identity`

You're done with Step 1 if:

- it prints JSON with `Account`, `Arn`, `UserId`
- no "Unable to locate credentials" error

Step 2: Create S3 bucket (model storage)

Why we are doing this

Your Lambda function does not permanently store large model files (`pump\_xgb\_model.json`, `training\_statistics.pkl`, `training\_report.json`) inside itself.

So we create an **S3 bucket** as the permanent storage location.

Later, Lambda will download these files from S3 during startup/inference.

Without this step:

- Lambda won't find model files
- You'll get errors like `NoSuchKey` or model loading failure

Do this:

Choose a unique bucket name (global uniqueness required), example:

``pump-anomaly-models-prod-2026``

Run: `aws s3 mb s3://pump-anomaly-models-jahnavi-2026 --region us-east-1`

Then verify:

```
aws s3 ls
...
```

Success looks like

You should see your bucket name in output, like:

- ``pump-anomaly-models-prod-2026``

Step 3: Upload model files to S3

Why we are doing this

Lambda will need these files:

- `pump\_xgb\_model.json` → trained model
- `training\_statistics.pkl` → stats for explainability/feature normalization
- `training\_report.json` → metadata/report info

Do this:

Go to your model files folder:

```
cd "path to models_ml"
```

Upload each file (replace bucket name with your actual one):

```
aws s3 cp pump_xgb_model.json s3://YOUR_BUCKET_NAME/  
aws s3 cp training_statistics.pkl s3://YOUR_BUCKET_NAME/  
aws s3 cp training_report.json s3://YOUR_BUCKET_NAME/  
aws s3 cp pump_xgb_results.pkl s3://YOUR_BUCKET_NAME/
```

Now verify upload:

```
aws s3 ls s3://YOUR_BUCKET_NAME/
```

Success looks like

You should see 3 files listed in bucket:

- `pump\_xgb\_model.json`
- `training\_statistics.pkl`
- `training\_report.json`
- pump_xgb_results.pkl

Step 4: Create Docker image

Do this exactly

Keep these values handy

- Region: `us-east-1`
- Account ID: `129874388416`
- S3 bucket: `pump-anomaly-models-prod-2026`

- ECR repo name: `pump-anomaly-detector`

1. Create ECR repo:

1. Open AWS Console in `us-east-1`
2. Go to ECR (Elastic Container Registry)
3. Click Create repository
4. Name: `pump-anomaly-detector`
5. Keep defaults, click Create

2. Create Dockerfile:

```
cat << 'EOF' > Dockerfile.lambda
FROM public.ecr.aws/lambda/python:3.11

# Set Numba cache directory (Lambda only allows writing to /tmp)
ENV NUMBA_CACHE_DIR=/tmp/numba_cache

# Create cache directory
RUN mkdir -p /tmp/numba_cache

# Install required system library for soundfile/librosa
RUN yum install -y libsndfile && yum clean all

# Upgrade pip and install dependencies
RUN pip install --no-cache-dir --upgrade pip && \
    pip install --no-cache-dir "numpy<2.0" && \
    pip install --no-cache-dir "numba==0.59.1" && \
    pip install --no-cache-dir \
    scipy==1.13.1 \
    scikit-learn==1.4.2 \
    soxr==0.3.7 \
    librosa==0.10.0 \
    soundfile==0.12.1 \
    xgboost==2.1.4 \
    boto3==1.28.25

# Copy Lambda handler and source code into container
COPY lambda_handler.py ${LAMBDA_TASK_ROOT}/
COPY src ${LAMBDA_TASK_ROOT}/src
COPY models_ml ${LAMBDA_TASK_ROOT}/models_ml
```

```
# Define the Lambda function entry point (handler)
CMD ["lambda_handler.lambda_handler"]
EOF
```

3. Login to ECR + build + push:

```
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin
12345678912.dkr.ecr.us-east-1.amazonaws.com
```

```
# Build Docker image
docker build \
  --no-cache \
  --platform linux/amd64 \
  -t 12345678912.dkr.ecr.us-east-1.amazonaws.com/pump-anomaly-detector:latest \
  -f Dockerfile.lambda .
```

```
# Push image to ECR
docker push 12345678912.dkr.ecr.us-east-1.amazonaws.com/pump-anomaly-detector:latest
```

```
# Update Lambda function
aws lambda update-function-code \
  --function-name pump-anomaly-detector-img \
  --image-uri 12345678912.dkr.ecr.us-east-1.amazonaws.com/pump-anomaly-detector:latest \
  --region us-east-1
```

Step 5: Create Lambda from this image

1. Go to Lambda → Create function
2. Select Container image
3. Function name: `pump-anomaly-detector-img`
4. Browse images:
 - Repository: `pump-anomaly-detector`
 - Tag: `latest`
5. Create function

5.1 Configure that new function
In `pump-anomaly-detector-img`:

- A) General config
- Memory: `1024 MB`
 - Timeout: `60 sec`

- Architecture: `x86\_64`

B) Environment variables

- `MODEL\_S3\_BUCKET` = `pump-anomaly-models-prod-2026`

- `LOG\_LEVEL` = `INFO`

C) Give Lambda permission to read your S3 bucket

Why: Right now Lambda has basic logging permissions only.

It still cannot download your model files from S3 unless we add `s3:GetObject` permission.

Without this, you'll get `AccessDenied` when the function runs.

6.1 Open Lambda role

1. In Lambda function page (`pump-anomaly-detector`)
2. Go to Configuration tab
3. Click Permissions
4. Under Execution role, click the role name (opens IAM in new tab)

6.2 Create inline policy

1. In IAM role page, click Add permissions
2. Click Create inline policy
3. Service: choose S3

6.3 Set allowed action

1. In Actions section
2. Expand Read
3. Check GetObject

6.4 Set resource ARN

1. In Resources section, choose Specific
2. Click Add ARN
3. Enter:
 - Bucket name: `pump-anomaly-models-prod-2026`
 - Object name: `\*`
4. It should resolve to:
`arn:aws:s3:::pump-anomaly-models-prod-2026/\*`

6.5 Save policy

1. Click Next

2. Policy name: `AllowReadPumpModels`
3. Click Create policy

Success check

In role permissions, you should now see policy `AllowReadPumpModels`.

Step 7: Test the new image Lambda before API Gateway

Why: We confirm function boots, imports packages, and reads model files from S3 before wiring any API.

7.1: Create API Gateway REST API.

Do this:

1. Open
<https://console.aws.amazon.com/apigateway/>
(region must be `us-east-1`)
2. Click Create API
3. Under REST API, click Build
4. Enter:
 - API name: `pump-anomaly-api`
 - Endpoint type: `Regional`
5. Click Create API

Success check:

You should land on the API page with the Resources panel showing root `/`.

7.2: Create `/predict` resource.

Do this:

1. In API Gateway, in the left Resources tree, click root `/`
2. Click Actions → Create Resource
3. Fill:
 - Resource Name: `predict`
 - Resource Path should auto-fill as: `/predict`
4. Click Create Resource

Success check:

You should now see `/predict` under `/` in the Resources tree.

7.3: Create a POST method for `/predict` and connect it to Lambda.

Do this:

1. In Resources tree, click `/predict`
2. Click Actions → Create Method
3. From dropdown, choose POST, then click checkmark
4. In setup screen, set:
 - Integration type: `Lambda Function`
 - Use Lambda Proxy integration: checked
 - Lambda Region: `us-east-1`
 - Lambda Function: `pump-anomaly-detector-img`
5. Click Save
6. If permission popup appears (“Add permission to Lambda function”), click OK

Success check

You should now see a POST method under `/predict`.

7.4 (optional but recommended) : Create `/health` endpoint so you can quickly check if API is alive.

Do this:

1. In Resources tree, click root `/`
2. Actions → Create Resource
3. Fill:
 - Resource Name: `health`
 - Resource Path: `/health`
4. Click Create Resource

Now create GET method:

5. Click `/health`
6. Actions → Create Method
7. Choose GET, click checkmark
8. Set integration:
 - Integration type: `Lambda Function`
 - Lambda proxy integration: checked
 - Region: `us-east-1`
 - Function: `pump-anomaly-detector-img`
9. Save
10. Click OK if permission prompt appears

Success check

You should see `GET` under `/health`.

7.5: Deploy API so your `/predict` and `/health` become live URLs.

Do this:

1. In API Gateway, click Actions → Deploy API
2. In popup:
 - If first deployment: choose [New Stage], stage name `prod`
 - If `prod` already exists: choose `prod`
3. Click Deploy

Success check

You should see an Invoke URL like:

`https://xxxxxxxxxx.execute-api.us-east-1.amazonaws.com/prod`

Copy this URL.

Enable CORS

1. Open API Gateway → your API `pump-anomaly-api`
2. In Resources tree, click `/predict`
3. Click Actions → Enable CORS
4. Keep defaults
5. Click Enable CORS and replace existing CORS headers
6. Confirm Yes, replace existing values

Repeat the same for `/health` (if you created it).

Important final step

After enabling CORS, you must deploy again:

1. Actions → Deploy API
2. Select stage `prod`
3. Click Deploy

Then CORS changes become live.

Step 8: Test with Postman

1. Open Postman.
2. Create a new request.
3. Set Method to `POST`.
4. Set URL to:
`https://xxxxxxxxxx.execute-api.us-east-1.amazonaws.com/prod/predict`

5. Go to Headers and add:
 - Key: `Content-Type`
 - Value: `application/octet-stream`
6. Go to Body → choose binary.
7. Select a `.wav` audio file.
8. Click Send.

Expected: JSON response with fields like prediction, confidence, etc.

FRONTEND:

Step 1: Set up Streamlit Cloud

1. Go to share.streamlit.io
2. Connect your GitHub account
3. Select your repo
4. Set main file path to `ui/app.py`
5. Click Deploy

Step 2: Add AWS Secrets

1. Click Manage app → Secrets
2. Add:

```
[aws]
AWS_ACCESS_KEY_ID = "your-key"
AWS_SECRET_ACCESS_KEY = "your-secret"
```

3. Click Save changes

Step 3: Update `config.py` In `api/src/config.py` change:

```
API_URL = "https://your-api-gateway-url.execute-api.us-east-1.amazonaws.com/prod"
```

Step 4: Check `app.py`

- `check_api_health()` — bypass for AWS URL
- `predict_audio()` — send raw bytes with audio/wav content type
- `get_training_report()` — fetch from S3 using Streamlit secrets

Step 5: Push to GitHub

```
git add .  
git commit -m "frontend: connect to AWS Lambda and S3"  
git push
```

Streamlit Cloud auto-deploys. Your public URL will be: <https://yourname.streamlit.app>